

Chapter 7. Agents II

[Chapter 6](#) introduced the *agent* architecture, the most powerful of the LLM architectures we have seen up until now. It is hard to overstate the potential of this combination of chain-of-thought prompting, tool use, and looping.

This chapter discusses two extensions to the agent architecture that improve performance for some use cases:

Reflection

Taking another page out of the repertoire of human thought patterns, this is about giving your LLM app the opportunity to analyze its past output and choices, together with the ability to remember reflections from past iterations.

Multi-agent

Much the same way as a team can accomplish more than a single person, there are problems that can be best tackled by teams of LLM agents.

Let's start with reflection.

Reflection

One prompting technique we haven't covered yet is *reflection* (also known as *self-critique*). *Reflection* is the creation of a loop between a creator prompt and a reviser prompt. This mirrors the creation process for many human-created artifacts, such as this chapter you're reading now, which is the result of a back and forth between the authors, reviewers, and editor until all are happy with the final product.

As with many of the prompting techniques we have seen so far, reflection can be combined with other techniques, such as chain-of-thought and tool calling. In this section, we'll look at reflection in isolation.

A parallel can be drawn to the modes of human thinking known as *System 1* (reactive or instinctive) and *System 2* (methodical and reflective), first introduced by Daniel Kahneman in the book *Thinking, Fast and Slow* (Farrar, Straus and Giroux, 2011). When applied correctly,

self-critique can help LLM applications get closer to something that resembles System 2 behavior ([Figure 7-1](#)).

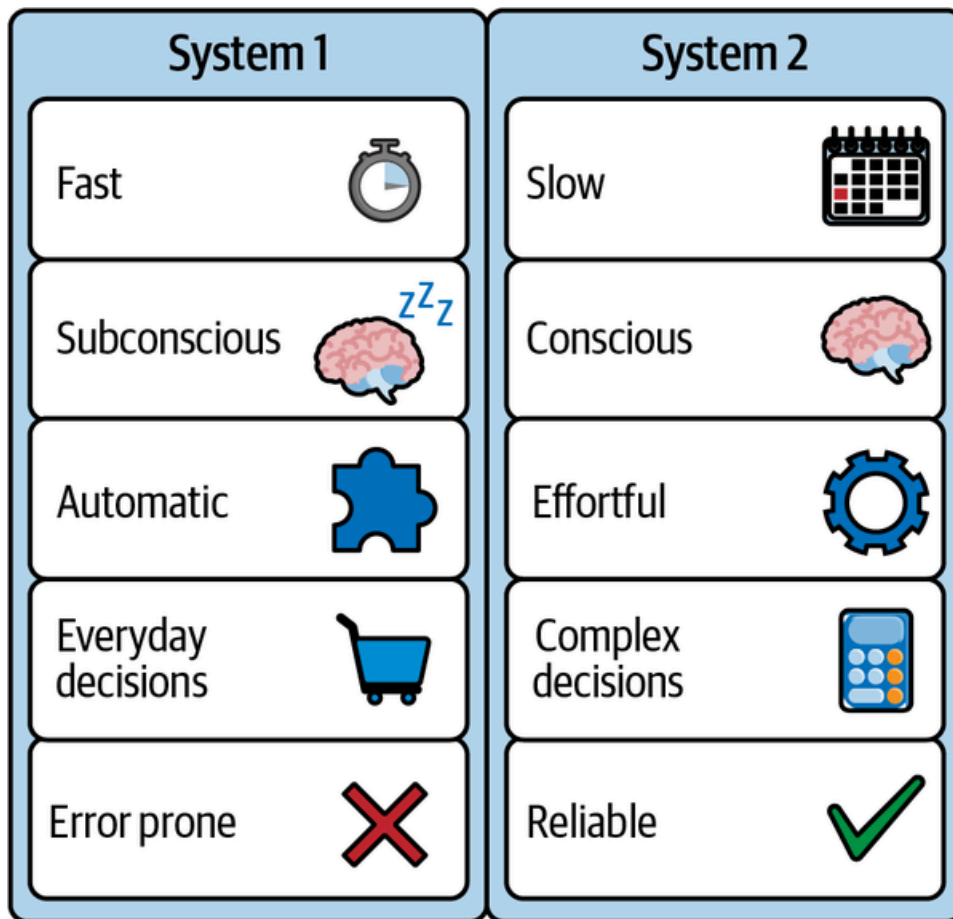


Figure 7-1. System 1 and System 2 thinking

We'll implement reflection as a graph with two nodes: `generate` and `reflect`. This graph will be tasked with writing three-paragraph essays, with the `generate` node writing or revising drafts of the essay, and `reflect` writing a critique to inform the next revision. We'll run the loop a fixed number of times, but a variation on this technique would be to have the `reflect` node decide when to finish. Let's see what it looks like:

Python

```
from typing import Annotated, TypedDict

from langchain_core.messages import (
    AIMessage,
    BaseMessage,
    HumanMessage,
    SystemMessage,
)
from langchain_openai import ChatOpenAI

from langgraph.graph import END, START, StateGraph
from langgraph.graph.message import add_messages

model = ChatOpenAI()
```

```

class State(TypedDict):
    messages: Annotated[list[BaseMessage], add_messages]

generate_prompt = SystemMessage(
    """You are an essay assistant tasked with writing excellent 3-paragraph
    essays."""
    "Generate the best essay possible for the user's request."
    """If the user provides critique, respond with a revised version of your
    previous attempts."""
)

def generate(state: State) -> State:
    answer = model.invoke([generate_prompt] + state["messages"])
    return {"messages": [answer]}

reflection_prompt = SystemMessage(
    """You are a teacher grading an essay submission. Generate critique and
    recommendations for the user's submission."""
    """Provide detailed recommendations, including requests for length, depth,
    style, etc."""
)

def reflect(state: State) -> State:
    # Invert the messages to get the LLM to reflect on its own output
    cls_map = {AIMessage: HumanMessage, HumanMessage: AIMessage}
    # First message is the original user request.
    # We hold it the same for all nodes
    translated = [reflection_prompt, state["messages"][0]] + [
        cls_map[msg.__class__](content=msg.content)
        for msg in state["messages"][1:]
    ]
    answer = model.invoke(translated)
    # We treat the output of this as human feedback for the generator
    return {"messages": [HumanMessage(content=answer.content)]}

def should_continue(state: State):
    if len(state["messages"]) > 6:
        # End after 3 iterations, each with 2 messages
        return END
    else:
        return "reflect"

builder = StateGraph(State)
builder.add_node("generate", generate)
builder.add_node("reflect", reflect)
builder.add_edge(START, "generate")
builder.add_conditional_edges("generate", should_continue)
builder.add_edge("reflect", "generate")

graph = builder.compile()

```

```

import {
  AIMessage,
  BaseMessage,
  SystemMessage,
  HumanMessage,
} from "@langchain/core/messages";
import { ChatOpenAI } from "@langchain/openai";
import {
  StateGraph,
  Annotation,
  messagesStateReducer,
  START,
  END,
} from "@langchain/langgraph";

const model = new ChatOpenAI();

const annotation = Annotation.Root({
  messages: Annotation({ reducer: messagesStateReducer, default: () => [] })
});

// fix multiline string
const generatePrompt = new SystemMessage(
  `You are an essay assistant tasked with writing excellent 3-paragraph essays.
  Generate the best essay possible for the user's request.
  If the user provides critique, respond with a revised version of your
  previous attempts.`
);

async function generate(state) {
  const answer = await model.invoke([generatePrompt, ...state.messages]);
  return { messages: [answer] };
}

const reflectionPrompt = new SystemMessage(
  `You are a teacher grading an essay submission. Generate critique and
  recommendations for the user's submission.
  Provide detailed recommendations, including requests for length, depth,
  style, etc.`
);

async function reflect(state) {
  // Invert the messages to get the LLM to reflect on its own output
  const clsMap: { [key: string]: new (content: string) => BaseMessage } = {
    ai: HumanMessage,
    human: AIMessage,
  };
  // First message is the original user request.
  // We hold it the same for all nodes
  const translated = [
    reflectionPrompt,
    state.messages[0],
    ...state.messages
      .slice(1)

```

```

        .map((msg) => new clsMap[msg._getType()](msg.content as string)),
    ];
    const answer = await model.invoke(translated);
    // We treat the output of this as human feedback for the generator
    return { messages: [new HumanMessage({ content: answer.content })] };
}

function shouldContinue(state) {
    if (state.messages.length > 6) {
        // End after 3 iterations, each with 2 messages
        return END;
    } else {
        return "reflect";
    }
}

const builder = new StateGraph(annotation)
    .addNode("generate", generate)
    .addNode("reflect", reflect)
    .addEdge(START, "generate")
    .addConditionalEdges("generate", shouldContinue)
    .addEdge("reflect", "generate");

const graph = builder.compile();

```

The visual representation of the graph is shown in [Figure 7-2](#).

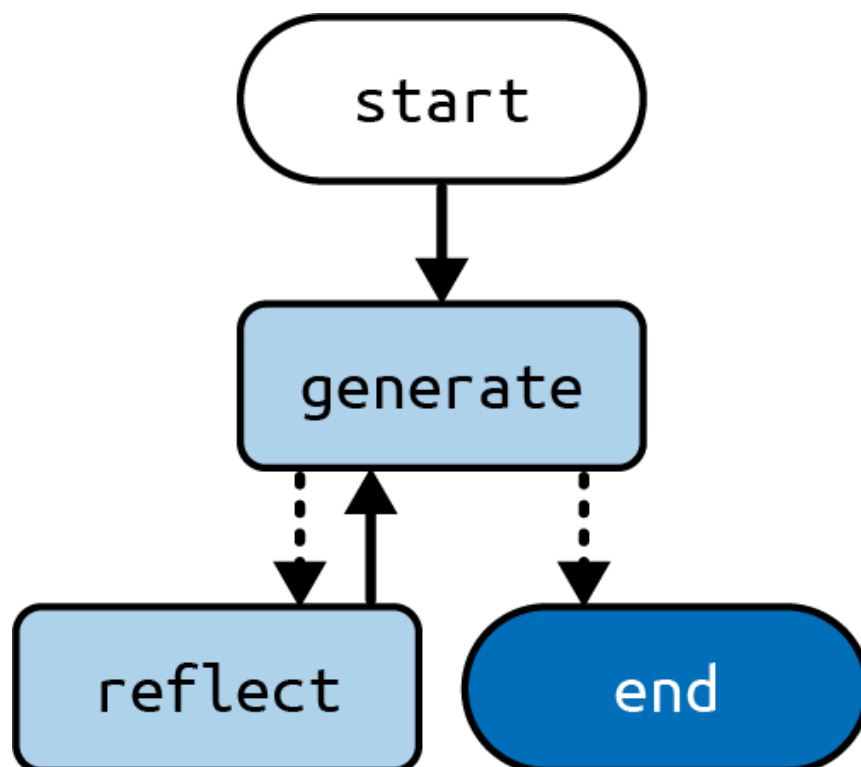


Figure 7-2. The reflection architecture

Notice how the `reflect` node tricks the LLM into thinking it is critiquing essays written by the user. And in tandem, the `generate` node is made to think that the critique comes from the user. This subterfuge is required because dialogue-tuned LLMs are trained on pairs of human-AI

messages, so a sequence of many messages from the same participant would result in poor performance.

One more thing to note: you might, at first glance, expect the end to come after a revise step, but in this architecture we have a fixed number of iterations of the `generate-reflect` loop; therefore we terminate after `generate` (so that the last set of revisions requested are dealt with). A variation on this architecture would instead have the `reflect` step make the decision to end the process (once it had no more comments).

Let's see what one of the critiques looks like:

```
{
  'messages': [
    HumanMessage(content='Your essay on the topicality of "The Little Prince" and its message in modern life is well-written and insightful. You have effectively highlighted the enduring relevance of the book's themes and its importance in today\'s society. However, there are a few areas where you could enhance your essay:\n\n1. Depth: While you touch upon the themes of cherishing simple joys, nurturing connections, and understanding human relationships, consider delving deeper into each of these themes. Provide specific examples from the book to support your points and explore how these themes manifest in contemporary life.\n\n2. Analysis: Consider analyzing how the book\'s messages can be applied to current societal issues or personal experiences. For instance, you could discuss how the Little Prince\'s perspective on materialism relates to consumer culture or explore how his approach to relationships can inform interpersonal dynamics in the digital age.\n\n3. Length: Expand on your ideas by adding more examples, discussing counterarguments, or exploring the cultural impact of "The Little Prince" in different parts of the world. This will enrich the depth of your analysis and provide a more comprehensive understanding of the book\'s relevance.\n\n4. Style: Your essay is clear and well-structured. To enhance the engagement of your readers, consider incorporating quotes from the book to illustrate key points or including anecdotes to personalize your analysis.\n\n5. Conclusion: Conclude your essay by summarizing the enduring significance of "The Little Prince" and how its message can inspire positive change in modern society. Reflect on the broader implications of the book\'s themes and leave the reader with a lasting impression.\n\nBy expanding on your analysis, incorporating more examples, and deepening your exploration of the book\'s messages, you can create a more comprehensive and compelling essay on the topicality of "The Little Prince" in modern life. Well done on your thoughtful analysis, and keep up the good work!', id='70c22b1d-ec96-4dc3-9fd0-d2c6463f9e2c'),
  ],
}
```

And the final output:

```
{
  'messages': [
    AIMessage(content='"The Little Prince" by Antoine de Saint-Exupéry
stands as a timeless masterpiece that continues to offer profou
insights into human relationships and values, resonating with
readers across generations. The narrative of the Little Prince\
travels and encounters with a myriad of characters serves as a
tapestry of allegorical representations, ....', response_metada
{'token_usage': {'completion_tokens': 420, 'prompt_tokens': 250
'total_tokens': 2921}, 'model_name': 'gpt-3.5-turbo',
'system_fingerprint': None, 'finish_reason': 'stop', 'logprobs'
None}, id='run-2e8f9f13-f625-4820-9c8b-b64e1c23daa2-0',
usage_metadata={'input_tokens': 2501, 'output_tokens': 420,
'total_tokens': 2921})),
  ],
}
```

This simple type of reflection can sometimes improve performance by giving the LLM multiple attempts at refining its output and by letting the reflection node adopt a different persona while critiquing the output.

There are several possible variations of this architecture. For one, we could combine the reflection step with the agent architecture of [Chapter 6](#), adding it as the last node right before sending output to the user. This would make the critique appear to come from the user, and give the application a chance to improve its final output without direct user intervention. Obviously this approach would come at the expense of higher latency.

In certain use cases, it could be helpful to ground the critique with external information. For instance, if you were writing a code-generation agent, you could have a step before `reflect` that would run the code through a linter or compiler and report any errors as input to `reflect`.

TIP

Whenever this approach is possible, we strongly recommend giving it a try, as it's likely to increase the quality of the final output.

Subgraphs in LangGraph

Before we dive into multi-agent architectures, let's look at an important technical concept in LangGraph that enables it. *Subgraphs* are graphs that are used as part of another graph. Here are some use cases for subgraphs:

- Building multi-agent systems (discussed in the next section).

- When you want to reuse a set of nodes in multiple graphs, you can define them once in a subgraph and then use them in multiple parent graphs.
- When you want different teams to work on different parts of the graph independently, you can define each part as a subgraph, and as long as the subgraph interface (the input and output schemas) is respected, the parent graph can be built without knowing any details of the subgraph.

There are two ways to add subgraph nodes to a parent graph:

Add a node that calls the subgraph directly

This is useful when the parent graph and the subgraph share state keys, and you don't need to transform state on the way in or out.

Add a node with a function that invokes the subgraph

This is useful when the parent graph and the subgraph have different state schemas, and you need to transform state before or after calling the subgraph.

Let's look at each in turn.

Calling a Subgraph Directly

The simplest way to create subgraph nodes is to attach a subgraph directly as a node. When doing so, it is important that the parent graph and the subgraph share state keys, because those shared keys will be used to communicate. (If your graph and subgraph do not share any keys, see the next section.)

NOTE

If you pass extra keys to the subgraph node (that is, in addition to the shared keys), they will be ignored by the subgraph node. Similarly, if you return extra keys from the subgraph, they will be ignored by the parent graph.

Let's see what it looks like in action:

Python

```
from langgraph.graph import START, StateGraph
from typing import TypedDict

class State(TypedDict):
    foo: str # this key is shared with the subgraph
```



```

class SubgraphState(TypedDict):
    foo: str # this key is shared with the parent graph
    bar: str

# Define subgraph
def subgraph_node(state: SubgraphState):
    # note that this subgraph node can communicate with the parent graph
    # via the shared "foo" key
    return {"foo": state["foo"] + "bar"}

subgraph_builder = StateGraph(SubgraphState)
subgraph_builder.add_node(subgraph_node)
...
subgraph = subgraph_builder.compile()

# Define parent graph
builder = StateGraph(State)
builder.add_node("subgraph", subgraph)
...
graph = builder.compile()

```

JavaScript

```

import { StateGraph, Annotation, START } from "@langchain/langgraph";

const StateAnnotation = Annotation.Root({
    foo: Annotation(),
});

const SubgraphStateAnnotation = Annotation.Root({
    // note that this key is shared with the parent graph state
    foo: Annotation(),
    bar: Annotation(),
});

// Define subgraph
const subgraphNode = async (state) => {
    // note that this subgraph node can communicate with
    // the parent graph via the shared "foo" key
    return { foo: state.foo + "bar" };
};

const subgraph = new StateGraph(SubgraphStateAnnotation)
    .addNode("subgraph", subgraphNode);
...
    .compile();

// Define parent graph
const parentGraph = new StateGraph(StateAnnotation)
    .addNode("subgraph", subgraph)
    .addEdge(START, "subgraph")

```

```
// Additional parent graph setup would go here
.compile();
```

Calling a Subgraph with a Function

You might want to define a subgraph with a completely different schema. In that case, you can create a node with a function that invokes the subgraph. This function will need to transform the input (parent) state to the subgraph state before invoking the subgraph and transform the results back to the parent state before returning the state update from the node.

Let's see what it looks like:

Python

```
class State(TypedDict):
    foo: str

class SubgraphState(TypedDict):
    # none of these keys are shared with the parent graph state
    bar: str
    baz: str

# Define subgraph
def subgraph_node(state: SubgraphState):
    return {"bar": state["bar"] + "baz"}

subgraph_builder = StateGraph(SubgraphState)
subgraph_builder.add_node(subgraph_node)
...
subgraph = subgraph_builder.compile()

# Define parent graph
def node(state: State):
    # transform the state to the subgraph state
    response = subgraph.invoke({"bar": state["foo"]})
    # transform response back to the parent state
    return {"foo": response["bar"]}

builder = StateGraph(State)
# note that we are using `node` function instead of a compiled subgraph
builder.add_node(node)
...
graph = builder.compile()
```

JavaScript

```
import { StateGraph, START, Annotation } from "@langchain/langgraph";

const StateAnnotation = Annotation.Root({
```

```

    foo: Annotation(),
  });

  const SubgraphStateAnnotation = Annotation.Root({
    // note that none of these keys are shared with the parent graph state
    bar: Annotation(),
    baz: Annotation(),
  });

  // Define subgraph
  const subgraphNode = async (state) => {
    return { bar: state.bar + "baz" };
  };

  const subgraph = new StateGraph(SubgraphStateAnnotation)
    .addNode("subgraph", subgraphNode);
  ...
  .compile();

  // Define parent graph
  const subgraphWrapperNode = async (state) => {
    // transform the state to the subgraph state
    const response = await subgraph.invoke({
      bar: state.foo,
    });
    // transform response back to the parent state
    return {
      foo: response.bar,
    };
  };

  const parentGraph = new StateGraph(StateAnnotation)
    .addNode("subgraph", subgraphWrapperNode)
    .addEdge(START, "subgraph")
    // Additional parent graph setup would go here
    .compile();

```

Now that we know how to use subgraphs, let's take a look at one of the big use cases for them: multi-agent architectures.

Multi-Agent Architectures

As LLM agents grow in size, scope, or complexity, several issues can show up and impact their performance, such as the following:

- The agent is given too many tools to choose from and makes poor decisions about which tool to call next ([Chapter 6](#) discussed some approaches to this problem).
- The context grows too complex for a single agent to keep track of; that is, the size of the prompts and the number of things they mention

grows beyond the capability of the model you're using.

- You want to use a specialized subsystem for a particular area, for instance, planning, research, solving math problems, and so on.

To tackle these problems, you might consider breaking your application into multiple smaller, independent agents and composing them into a multi-agent system. These independent agents can be as simple as a prompt and an LLM call or as complex as a ReAct agent (introduced in [Chapter 6](#)). [Figure 7-3](#) illustrates several ways to connect agents in a multi-agent system.

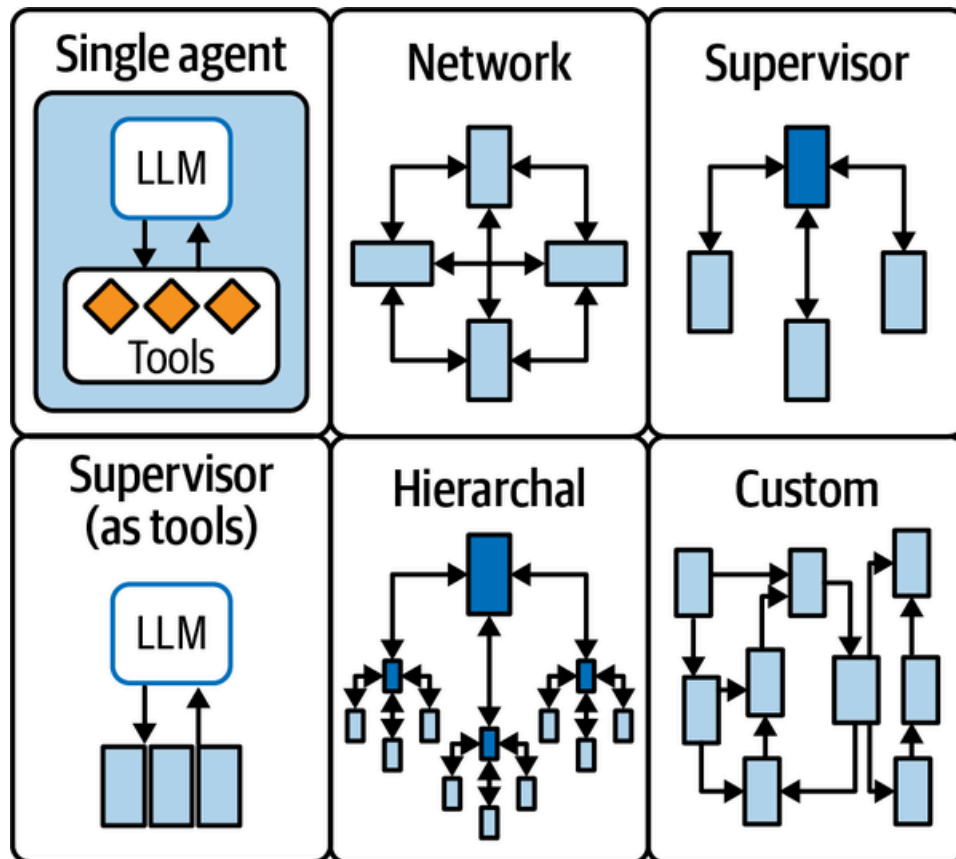


Figure 7-3. Multiple strategies for coordinating multiple agents

Let's look at [Figure 7-3](#) in more detail:

Network

Each agent can communicate with every other agent. Any agent can decide which other agent is to be executed next.

Supervisor

Each agent communicates with a single agent, called the *supervisor*. The supervisor agent makes decisions on which agent (or agents) should be called next. A special case of this architecture implements the supervisor agent as an LLM call with tools, as covered in [Chapter 6](#).

Hierarchical

You can define a multi-agent system with a supervisor of supervisors. This is a generalization of the supervisor architecture and allows for more complex control flows.

Custom multi-agent workflow

Each agent communicates with only a subset of agents. Parts of the flow are deterministic, and only select agents can decide which other agents to call next.

The next section dives deeper into the supervisor architecture, which we think has a good balance of capability and ease of use.

Supervisor Architecture

In this architecture, we add each agent to the graph as a node and also add a supervisor node, which decides which agents should be called next. We use conditional edges to route execution to the appropriate agent node based on the supervisor's decision. Refer back to [Chapter 5](#) for an introduction to LangGraph, which goes over the concepts of nodes, edges, and more.

Let's first see what the supervisor node looks like:

Python

```
from typing import Literal
from langchain_openai import ChatOpenAI
from pydantic import BaseModel

class SupervisorDecision(BaseModel):
    next: Literal["researcher", "coder", "FINISH"]

model = ChatOpenAI(model="gpt-4o", temperature=0)
model = model.with_structured_output(SupervisorDecision)

agents = ["researcher", "coder"]

system_prompt_part_1 = f"""You are a supervisor tasked with managing a
conversation between the following workers: {agents}. Given the following u
request, respond with the worker to act next. Each worker will perform a
task and respond with their results and status. When finished,
respond with FINISH."""

system_prompt_part_2 = f"""Given the conversation above, who should act nex
should we FINISH? Select one of: {' '.join(agents)}, FINISH"""

def supervisor(state):
    messages = [
        ("system", system_prompt_part_1),
        *state["messages"],
```

```

        ( "system",      system_prompt_part_2)
    ]
    return model.invoke(messages)

```

JavaScript

```

import { ChatOpenAI } from 'langchain-openai';
import { z } from 'zod';

const SupervisorDecision = z.object({
  next: z.enum([ 'researcher', 'coder', 'FINISH' ]),
});

const model = new ChatOpenAI({ model: 'gpt-4o', temperature: 0 });
const modelWithStructuredOutput = model.withStructuredOutput(SupervisorDecision);

const agents = [ 'researcher', 'coder' ];

const systemPromptPart1 = `You are a supervisor tasked with managing a
conversation between the following workers: ${agents.join(', ')}. Given the
following user request, respond with the worker to act next. Each worker
will perform a task and respond with their results and status. When
finished, respond with FINISH.`;

const systemPromptPart2 = `Given the conversation above, who should act next?
Should we FINISH? Select one of: ${agents.join(', ')}, FINISH`;

const supervisor = async (state) => {
  const messages = [
    { role: 'system', content: systemPromptPart1 },
    ...state.messages,
    { role: 'system', content: systemPromptPart2 }
  ];

  return await modelWithStructuredOutput.invoke({ messages });
};

```

NOTE

The code in the prompt requires the names of your subagents to be self-explanatory and distinct. For instance, if they were simply called `agent_1` and `agent_2`, the LLM would have no information to decide which one is appropriate for each task. If needed, you could modify the prompt to add a description of each agent, which could help the LLM in picking an agent for each query.

Now let's see how to integrate this supervisor node into a larger graph that includes two other subagents, which we will call researcher and coder. Our overall goal with this graph is to handle queries that can be answered either by the researcher by itself or the coder by itself, or even

both of them in succession. This example doesn't include implementations for either the researcher or coder—the key idea is they could be any other LangGraph graph or node:

Python

```
from typing import Literal
from langchain_openai import ChatOpenAI
from langgraph.graph import StateGraph, MessagesState, START

model = ChatOpenAI()

class AgentState(BaseModel):
    next: Literal["researcher", "coder", "FINISH"]

def researcher(state: AgentState):
    response = model.invoke(...)
    return {"messages": [response]}

def coder(state: AgentState):
    response = model.invoke(...)
    return {"messages": [response]}

builder = StateGraph(AgentState)
builder.add_node(supervisor)
builder.add_node(researcher)
builder.add_node(coder)

builder.add_edge(START, "supervisor")
# route to one of the agents or exit based on the supervisor's decision
builder.add_conditional_edges("supervisor", lambda state: state["next"])
builder.add_edge("researcher", "supervisor")
builder.add_edge("coder", "supervisor")

supervisor = builder.compile()
```

JavaScript

```
import {
  StateGraph,
  Annotation,
  MessagesAnnotation,
  START,
  END,
} from "@langchain/langgraph";
import { ChatOpenAI } from "@langchain/openai";

const model = new ChatOpenAI({
  model: "gpt-4o",
});
```

```

const StateAnnotation = Annotation.Root({
  ...MessagesAnnotation.spec,
  next: Annotation(),
});

const researcher = async (state) => {
  const response = await model.invoke(...);
  return { messages: [response] };
};

const coder = async (state) => {
  const response = await model.invoke(...);
  return { messages: [response] };
};

const graph = new StateGraph(StateAnnotation)
  .addNode("supervisor", supervisor)
  .addNode("researcher", researcher)
  .addNode("coder", coder)
  .addEdge(START, "supervisor")
  // route to one of the agents or exit based on the supervisor's decision
  .addConditionalEdges("supervisor", async (state) =>
    state.next === 'FINISH' ? END : state.next)
  .addEdge("researcher", "supervisor")
  .addEdge("coder", "supervisor")
  .compile();

```

A few things to notice: In this example, both subagents (researcher and coder) can see each other's work, as all progress is recorded in the messages list. This isn't the only way to organize this. Each of the subagents could be more complex. For instance, a subagent could be its own graph that maintains internal state and only outputs a summary of the work it did.

After each agent executes, we route back to the supervisor node, which decides if there is more work to be done and which agent to delegate that to if so. This routing isn't a hard requirement for this architecture; we could have each subagent make a decision as to whether its output should be returned directly to the user. To do that, we'd replace the hard edge between, say, researcher and supervisor, with a conditional edge (which would read some state key updated by researcher).

Summary

This chapter covered two important extensions to the agent architecture: reflection and multi-agent architectures. The chapter also looked at how to work with subgraphs in LangGraph, which are a key building block for multi-agent systems.

These extensions add more power to the LLM agent architecture, but they shouldn't be the first thing you reach for when creating a new agent. The best place to start is usually the straightforward architecture we discussed in [Chapter 6](#).

[Chapter 8](#) returns to the trade-off between reliability and agency, which is the key design decision when building LLM apps today. This is especially important when using the agent or multi-agent architectures, as their power comes at the expense of reliability if left unchecked. After diving deeper into why this trade-off exists, [Chapter 8](#) will cover the most important techniques at your disposal to navigate that decision, and ultimately improve your LLM applications and agents.

Table of contents collapsed